

LABORATORIO II- AUTOMATIZACIÓN 2

I. DEEP LEARNIG

En este taller pondrás en práctica todo el proceso de construcción de un modelo de Deep Learning para el análisis de imágenes médicas, utilizando el dataset público de rayos X de tórax disponible en Kaggle:

<https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

Crea una sección en la wiki de Github dándole un nombre a tu proyecto y presentando a los desarrolladores y en que consiste tu proyecto

Actividades a desarrollar:

1. Exploración y análisis del dataset

- Revisa la estructura de las carpetas, cantidad de imágenes disponibles por clase y formato de datos.
- Realiza un breve análisis exploratorio: ejemplos de imágenes, distribución de clases y posibles retos del dataset (desbalance, resolución, etc.).

Crea una sección en la wiki de Github llamada 'Análisis del problema', donde quede consignada esta información

2. Concepto e implementación de DataLoader

- Investiga qué es un DataLoader en PyTorch/TensorFlow y cuál es su función en el entrenamiento de modelos.
- Explica cómo se relaciona el batch_size con el DataLoader y con el proceso de optimización.
- Implementa y prueba tu propio DataLoader para el entrenamiento de este dataset. Realiza verificaciones iniciales mostrando imágenes por lotes para confirmar que el DataLoader funciona correctamente.

3. Diseño de la arquitectura de la Red Convolucional (CNN)

- Investiga diferentes arquitecturas de CNN aplicadas a imágenes médicas (ej. LeNet, VGG, ResNet adaptadas).
- Define y justifica tu propuesta de arquitectura propia.

Representa gráficamente la arquitectura (diagrama de bloques que muestre capas convolucionales, activaciones, pooling, fully connected, etc.).

Crea una sección en la wiki de Github llamada 'Diseño de solución', donde quede consignada esta información

4. Entrenamiento del modelo

- Configura el proceso de entrenamiento (definición de función de pérdida, optimizador, métricas de evaluación y número de épocas).
- Entrena tu CNN con los datos y registra la evolución de la pérdida y la precisión en entrenamiento y validación.

5. Evaluación de resultados

- Evalúa el desempeño del modelo en el conjunto de prueba.
- Presenta métricas relevantes (accuracy, matriz de confusión, precision, recall, F1-score).
- Incluye visualizaciones que apoyen tu análisis (curvas de entrenamiento, imágenes mal clasificadas, etc.).

Crea una sección en la wiki de Github llamada 'Resultados', donde quede consignada esta información

II. SISTEMA RAG

Integre un sistema RAG experto en neumonía a su sistema, donde un usuario pueda hacer preguntas acerca de esta patología y el sistema responda de forma asertiva.

Investigue y pruebe varios modelos de embeebing y modelos the question-answering desde Hugging-Face.

Crea una sección en la wiki de Github llamada 'Sistema RAG, donde quede consignada los modelos de embeebing y de LLM trabajados, la razón por a que se descartaron, y cual se selecciono finalmente y el porque.

Producto final esperado:

Envíe su proyecto de GitHub con la siguiente información

- Notebook o script bien documentado con todo el flujo de trabajo, y su sistema RAG implementado
- Explicación escrita de la arquitectura diseñada y su justificación.
- Gráfico de la arquitectura de la red convolucional.
- Resultados de entrenamiento y análisis de desempeño del modelo.
- Registro de experimentos del sistema RAG